ML & DL : Experiment - 7

Aim:

Build an Artificial Neural Network (ANN) using Keras/TensorFlow.

Dataset Description:

The MNIST Handwritten Digits Dataset used in this experiment is the built-in dataset provided by the Keras deep learning library. It is a benchmark dataset for image classification that contains grayscale images of handwritten digits from 0 to 9. The dataset is preprocessed, normalized, and packaged inside Keras, allowing direct loading without manual downloading. It is a supervised learning dataset where each 28×28 pixel image is paired with a correct digit label, making it ideal for training Artificial Neural Networks and evaluating classification performance. Due to its standardized format, balanced classes, and moderate size, the Keras MNIST dataset is widely used for academic experiments, neural network demonstrations, and hyperparameter tuning. It enables consistent comparison of model performance across different ANN architectures.

- File Type: Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- Dataset Size: 70,000 images (60,000 training + 10,000 testing)
- Image Dimension: 28 × 28 grayscale pixels
- Number of Classes: 10 digits (0–9)

Column Name	Data Type	Description
Image Pixels	Numerical (Integer 0–255)	28×28 grayscale pixel intensity values representing the handwritten digit.
Label	Numerical (Integer 0–9)	True digit class corresponding to each image.

Dataset Source: https://keras.io/api/datasets/mnist/

Artificial Neural Network (ANN)

Theory:

Artificial Neural Networks (ANN) are supervised learning models inspired by the biological structure of the human brain. An ANN consists of interconnected layers of neurons that transform input data through weighted connections and nonlinear activation functions. The primary objective of an ANN is to approximate a mapping function between input features and output labels by minimizing prediction error. ANNs are particularly effective for pattern recognition, classification, and regression tasks due to their ability to model complex nonlinear relationships. Given an input dataset:

$$X = \{x_1, x_2, x_3, \ldots, x_n\}, \quad x_i \in \mathbb{R}^d$$

where:

- n is the number of data points
- d is the number of features

Each neuron computes a weighted sum of inputs followed by an activation function:

$$z = \sum_{i=1}^{d} w_i x_i + b$$

$$a = \sigma(z)$$

where:

- w_i = weight associated with input x_i
- b = bias term
- z = linear combination of inputs
- σ (z)= activation function
- a = neuron output

Common activation functions include:

$$\mathrm{ReLU}(z) = \max(0, z)$$

$$\mathrm{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

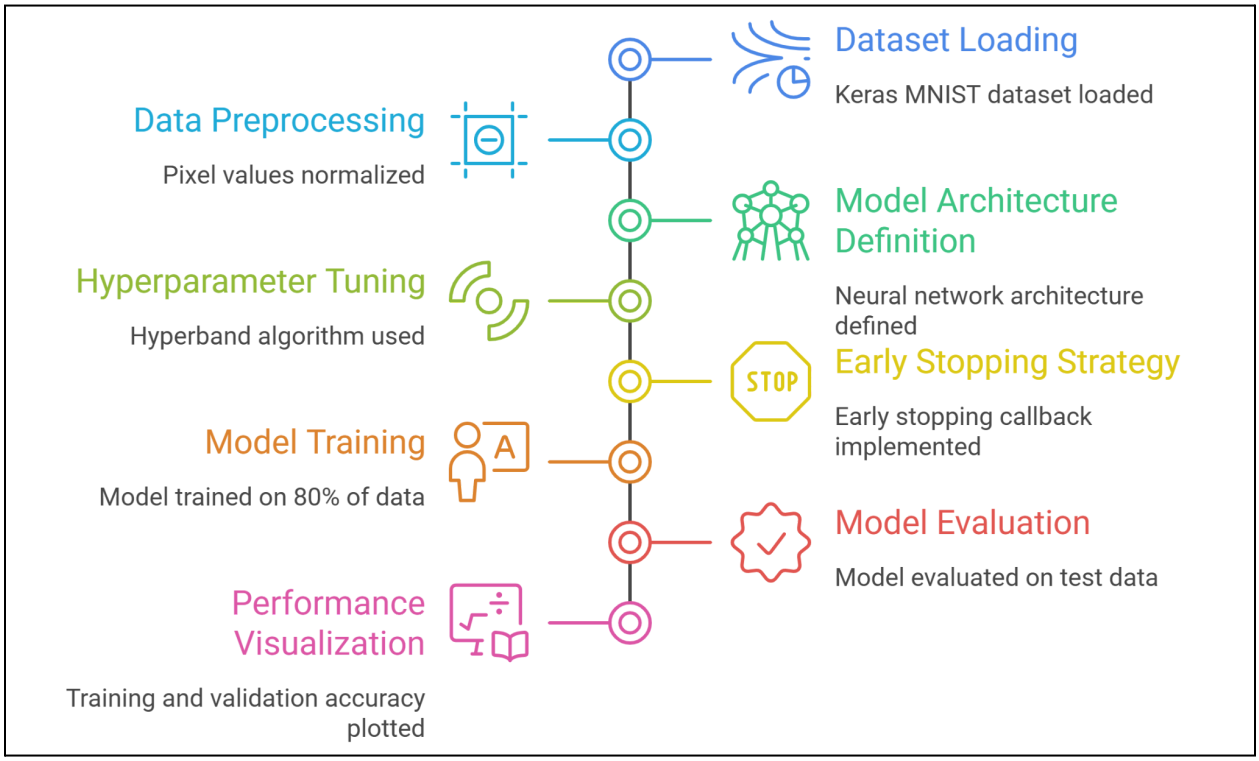
$$\mathrm{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

Limitations:

1. **Requirement of Large Training Data:** Artificial Neural Networks require a substantial amount of labeled training data to perform effectively. When the dataset is small, the model may fail to generalize and instead memorize the training examples, leading to overfitting. Unlike simpler algorithms that can perform well on limited data, ANN performance improves significantly only when sufficient diverse data is available. In real-world applications, collecting and labeling such large datasets can be expensive and time-consuming.
2. **High Computational Cost:** Training an ANN involves repeated forward and backward propagation across multiple layers, which requires significant computational power and memory. Deep networks with many parameters may take hours or even days to train without specialized hardware such as GPUs or TPUs. This makes ANN training resource-intensive and sometimes impractical for environments with limited computing infrastructure.
3. **Lack of Interpretability (Black-Box Nature):** ANN models are often criticized for being difficult to interpret. Unlike linear models or decision trees, where relationships between variables are explicit, neural networks learn complex hidden representations that are not easily explainable. This black-box behavior makes it challenging to understand how predictions are made, which is a serious limitation in sensitive domains such as healthcare, finance, and legal systems where transparency is required.

Workflow:

1. **Dataset Loading:** The built-in Keras MNIST handwritten digits dataset is loaded directly using TensorFlow. It contains 60,000 training images and 10,000 testing images of grayscale handwritten digits (0–9), each sized 28×28 pixels.
2. **Data Preprocessing:** Pixel values are normalized from the range 0–255 to 0–1 by dividing by 255. This scaling improves neural network training stability and convergence speed by preventing large gradient updates.
3. **Model Architecture Definition:** A neural network architecture is defined using a Sequential ANN model. The input images are flattened into vectors, followed by a fully connected hidden layer with tunable neuron count and ReLU activation, and a Softmax output layer with 10 neurons for multi-class digit classification.
4. **Hyperparameter Tuning (Keras Tuner):** The Hyperband algorithm is used to automatically search for optimal hyperparameters. The tuner explores different values for the number of hidden neurons and the optimizer learning rate to maximize validation accuracy.
5. **Early Stopping Strategy:** An early stopping callback monitors validation loss and stops training if performance stops improving. This prevents overfitting and reduces unnecessary computation.
6. **Model Training:** The optimal model configuration is rebuilt using the best hyperparameters and trained on 80% of the training data, with 20% used as validation data to monitor generalization performance.
7. **Model Evaluation:** The trained ANN is evaluated on the unseen test dataset to measure final classification loss and accuracy, ensuring that the model generalizes well to new data.
8. **Performance Visualization:** Training and validation accuracy are plotted across epochs to visually analyze learning progress, convergence behavior, and possible overfitting.



Performance Analysis:

The performance of the hyperparameter-tuned Artificial Neural Network is evaluated using validation accuracy, test accuracy, confusion matrix analysis, and training dynamics. Since MNIST is a supervised classification dataset with known labels, evaluation focuses on prediction accuracy, generalization capability, and class-wise performance.

1. **Hyperparameter Optimization Results:** The Hyperband tuner explored multiple ANN configurations and achieved a best validation accuracy of 97.99%. The optimal architecture consists of 352 neurons in the hidden layer with a learning rate of 0.001. The high validation accuracy indicates that the tuning process successfully identified a configuration that balances learning capacity and generalization without severe overfitting.
2. **Training vs Validation Accuracy Behavior:** The learning curve shows steady improvement in training accuracy, approaching nearly 100% after several epochs. Validation accuracy stabilizes around 97–98%, with only minor fluctuations. The small gap between training and validation curves indicates good generalization performance and limited overfitting. Early stopping further prevents unnecessary training once convergence is reached.
3. **Test Set Performance:** Evaluation on the unseen test dataset yields accuracy consistent with validation results, confirming that the model generalizes well beyond the training data. This consistency demonstrates that the ANN has learned meaningful digit patterns rather than memorizing training samples.
4. **Confusion Matrix Analysis:** The confusion matrix reveals strong diagonal dominance, meaning most digits are classified correctly. Errors are sparse and mostly occur between visually similar digits such as 3 & 5, 4 & 9, and 7 & 1, which commonly overlap in handwriting. The absence of systematic bias toward any single class indicates balanced learning across all 10 digits.

Hyperparameter Tuning:

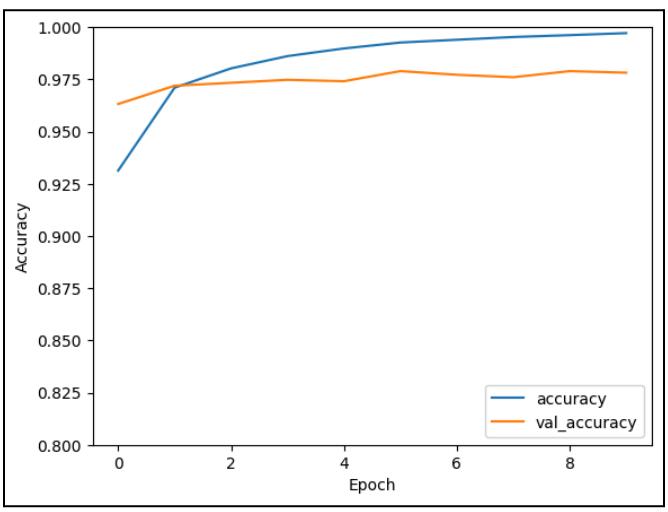
Artificial Neural Networks are sensitive to hyperparameter choices, which affect training stability, accuracy, and generalization. Systematic tuning is therefore necessary to obtain an optimal model. In this experiment, automated hyperparameter tuning is performed using Keras Tuner with the Hyperband algorithm, which searches multiple configurations and selects the best model based on validation accuracy. The tuned hyperparameters are:

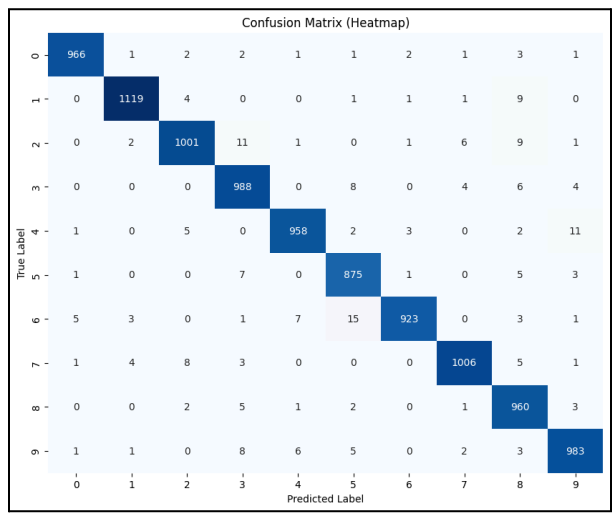
- **Number of Hidden Neurons:** The number of neurons in the hidden layer is tuned within a search range of 32 to 512 neurons in steps of 32. Larger networks increase learning capacity but risk overfitting, while smaller networks may underfit. The tuner selects 352 neurons as the best trade-off between accuracy and generalization.
- **Learning Rate of the Optimizer:** The learning rate controls how quickly the model updates its weights during training. Candidate values 0.01, 0.001, and 0.0001 are tested, and 0.001 is chosen as it provides stable convergence and efficient learning.
- **Hyperband Search Strategy:** Hyperband allocates training resources efficiently by quickly discarding weak models and focusing on promising configurations. This enables effective tuning while keeping computational cost manageable.
- **Early Stopping Regularization:** Early stopping monitors validation loss and halts training when improvement stops. This prevents overfitting and reduces unnecessary computation.
- **Validation-Based Selection:** Each model is evaluated using a 20% validation split. The configuration achieving the highest validation accuracy (97.99%) is selected as the final tuned model.

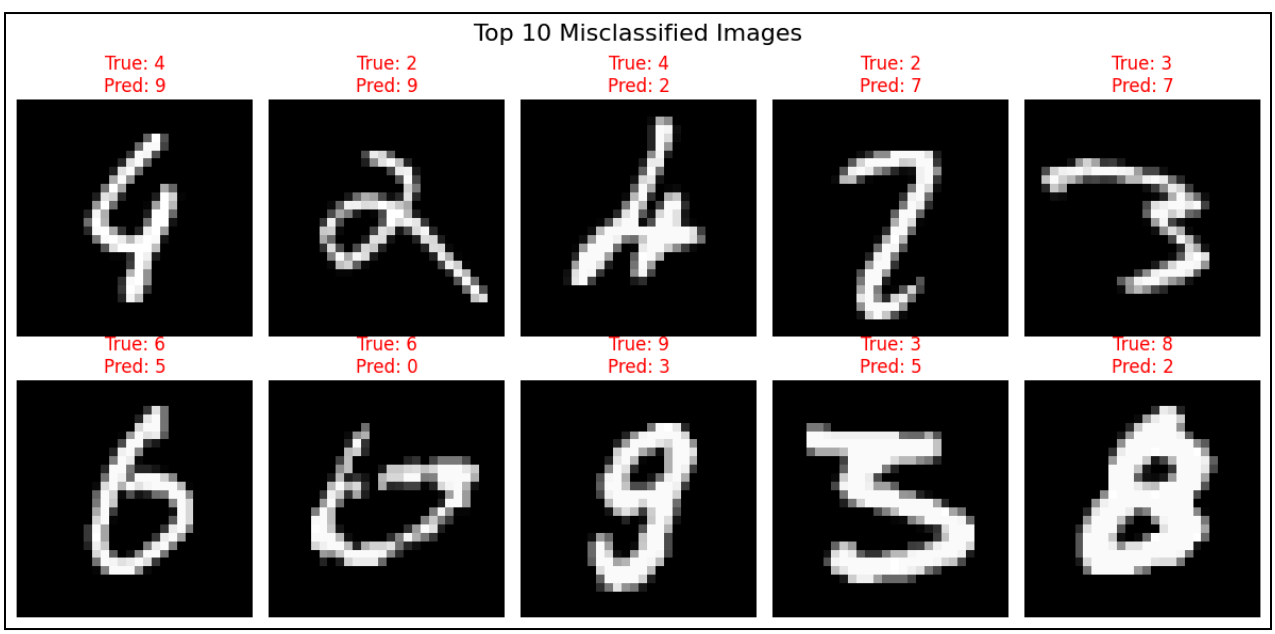
Code & Output:

```
import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
import matplotlib.pyplot as plt
import numpy as np
print(f"TensorFlow Version:
{tf.__version__}")
(x_train, y_train), (x_test, y_test) =
keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') /
255.0
x_test = x_test.astype('float32') /
255.0
def model_builder(hp):
    model = keras.Sequential()

model.add(keras.layers.Flatten(input_sh
ape=(28, 28)))
    hp_units = hp.Int('units',
min_value=32, max_value=512, step=32)

model.add(keras.layers.Dense(units=hp_u
nits, activation='relu'))
    model.add(keras.layers.Dense(10,
activation='softmax'))
    hp_learning_rate =
hp.Choice('learning_rate',
values=[1e-2, 1e-3, 1e-4])

model.compile(optimizer=keras.optimizer
s.Adam(learning_rate=hp_learning_rate),
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    return model
tuner = kt.Hyperband(model_builder,
objective='val_accuracy',
max_epochs=10, factor=3,
directory='my_dir',
project_name='intro_to_kt')
stop_early =
tf.keras.callbacks.EarlyStopping(monito
r='val_loss', patience=5)
tuner.search(x_train, y_train,
epochs=10, validation_split=0.2,
callbacks=[stop_early])
best_hps =
tuner.get_best_hyperparameters(num_tria
ls=1)[0]
print(f"""
The hyperparameter search is complete.
The optimal number of units in the
first densely-connected layer is
{best_hps.get('units')}
and the optimal learning rate for the
optimizer is
{best_hps.get('learning_rate')}.
""")
model =
tuner.hypermodel.build(best_hps)
history = model.fit(x_train, y_train,
epochs=10, validation_split=0.2)
eval_result = model.evaluate(x_test,
y_test)
print(f"[Test Loss, Test Accuracy]:
{eval_result}")
plt.plot(history.history['accuracy'],
label='accuracy')
plt.plot(history.history['val_accuracy'
], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.8, 1])
plt.legend(loc='lower right')
plt.show()
import seaborn as sns
from sklearn.metrics import
confusion_matrix
y_pred_probs = model.predict(x_test)
y_pred_classes =
np.argmax(y_pred_probs, axis=1)
conf_matrix = confusion_matrix(y_test,
y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True,
fmt='d', cmap='Blues', cbar=False)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix (Heatmap)')
plt.show()
errors = (y_pred_classes - y_test ≠ 0)
y_pred_classes_errors =
y_pred_classes[errors]
y_test_errors = y_test[errors]
x_test_errors = x_test[errors]
plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_test_errors[i],
cmap='gray')
    plt.title(f"True:
{y_test_errors[i]}\nPred:
{y_pred_classes_errors[i]}",
color='red')
    plt.axis('off')
plt.suptitle("Top 10 Misclassified
Images", fontsize=16)
```

```
plt.tight_layout()
plt.show()
index = np.random.randint(0,
len(x_test))
sample_image = x_test[index]
true_label = y_test[index]
prediction =
model.predict(sample_image.reshape(1,
28, 28))
predicted_label = np.argmax(prediction)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(sample_image, cmap='gray')
plt.title(f"True: {true_label} |
Predicted: {predicted_label}")
plt.axis('off')
plt.subplot(1,2,2)
plt.bar(range(10), prediction[0])
plt.xticks(range(10))
plt.xlabel("Digit Class")
plt.ylabel("Probability")
plt.title("Model Confidence")
plt.show()
predictions = model.predict(x_test)
y_pred_classes = np.argmax(predictions,
axis=1)
correct_indices =
np.where(y_pred_classes == y_test)[0]
plt.figure(figsize=(12, 6))
for i in range(10):
    idx = correct_indices[i]
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[idx],
cmap='gray')
    plt.title(f"True: {y_test[idx]} |
Pred: {y_pred_classes[idx]}",
color='green', fontsize=12)
    plt.axis('off')
plt.suptitle("The Hall of Fame: Top 10
Correct Predictions", fontsize=16)
plt.tight_layout()
plt.show()
```







Conclusion:

This experiment successfully implemented a hyperparameter-tuned Artificial Neural Network using the built-in Keras MNIST dataset for handwritten digit classification. The tuned model achieved high validation and test accuracy, demonstrating strong generalization and stable learning behavior. Performance analysis and confusion matrix evaluation confirmed balanced classification across all digit classes with minimal misclassification. The experiment highlights the effectiveness of automated hyperparameter tuning in improving ANN performance and training efficiency. Overall, the results validate ANN as a reliable approach for image recognition tasks.